SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105.

CSA0619-DESIGN AND ANALYSIS OF ALGORITHM

Algorithm Comparison Using Graphical Representation
Comparison of $O(n^2)$ and $O(n \log n)$ Algorithms

Presented By:
M. Yasodha Krishna
192472100

Introduction

- An algorithm is a step-by-step procedure used to solve a problem.
- Different algorithms require different amounts of time to complete the same task.
- Time complexity helps measure the efficiency of an algorithm as the input size increases.
- In this presentation, Bubble Sort and Merge Sort are compared based on their execution time and performance.
- The comparison is shown using tables and graphs.

Problem Statement

Problem Statement
The objective is to compare the performance of two sorting algorithms:
Bubble Sort with time complexity **$O(n^2)$**
Merge Sort with time complexity **O(n log n)**
using graphical representation.
Dataset
Random integer arrays
Input sizes:

- 100
- 500
- 1000
- 2000

Assumptions
Both algorithms use the **same input dataset**.
The comparison is performed on the **same computer**.
Python programming language is used.
Execution time is measured in seconds.

Algorithm / Pseudocode

Bubble Sort ($O(n^2)$)

```
BubbleSort(A)

for i = 0 to n-1
    for j = 0 to n-i-2
        if A[j] > A[j+1]
            swap(A[j], A[j+1])

return A
```

Time Complexity

- Best Case: O(n)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

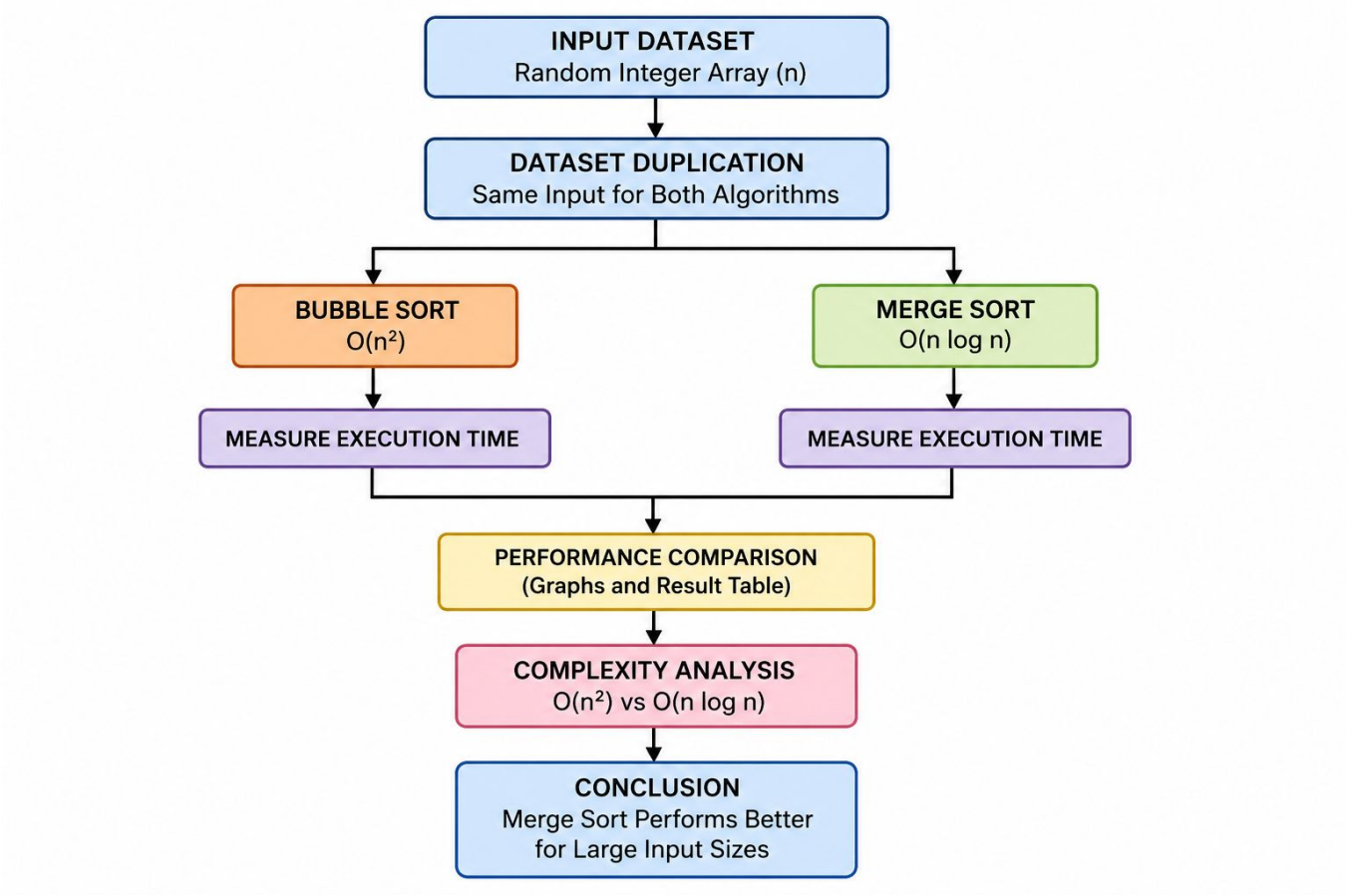
Merge Sort (O(n log n))

```
MergeSort(A)

if size ≤ 1
    return A

Divide array into two halves

Recursively sort left half

Recursively sort right half

Merge the sorted halves

return sorted array
```

Time Complexity

- Best Case: **O(n log n)**
- Average Case: **O(n log n)**
- Worst Case: **O(n log n)**

Architecture Diagram



Implementation (Code)

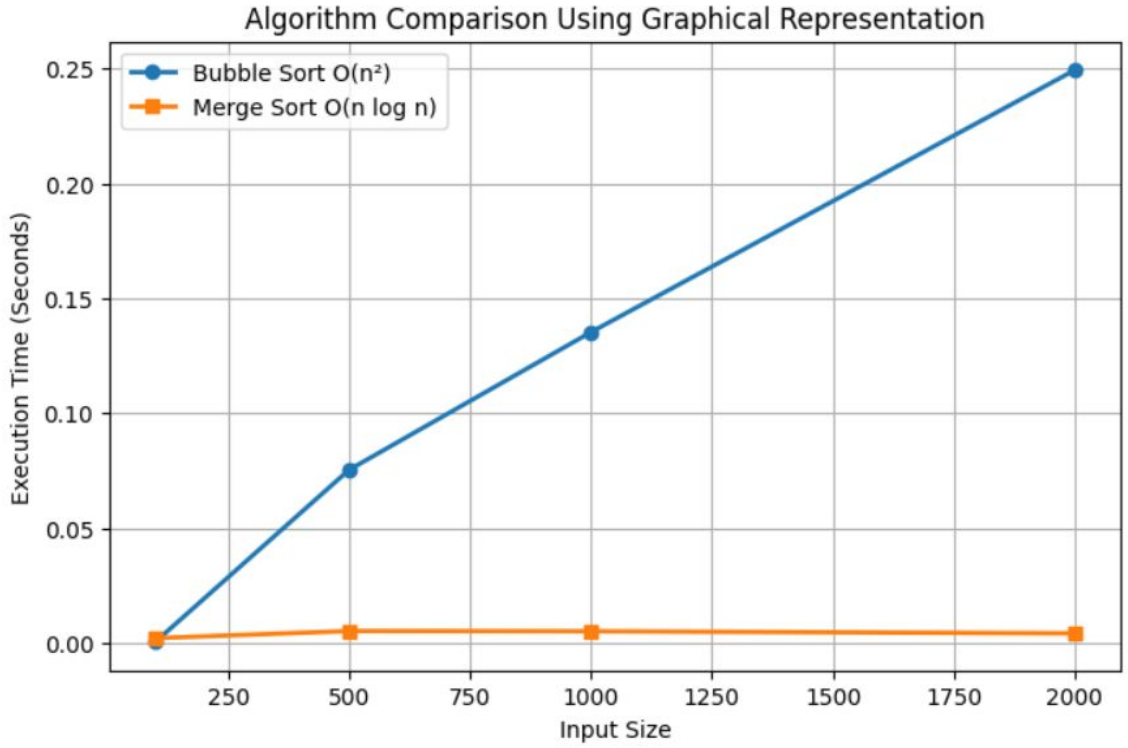
```python
import random
import time
import matplotlib.pyplot as plt

def bubble_sort(arr):
    arr = arr.copy()
    n = len(arr)

    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr


def merge_sort(arr):

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    merged = []

    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])

    return merged


sizes = [100, 500, 1000, 2000]

bubble_times = []
merge_times = []

print("=" * 60)
print("Algorithm Comparison: Bubble Sort vs Merge Sort")
print("=" * 60)

print(f"{'Input Size':<15}{'Bubble Sort':<20}{'Merge Sort':<20}")

for size in sizes:

    data = [random.randint(1, 10000) for _ in range(size)]

    start = time.perf_counter()
    bubble_sort(data)
    end = time.perf_counter()

    bubble_time = end - start
    bubble_times.append(bubble_time)

    start = time.perf_counter()
    merge_sort(data)
    end = time.perf_counter()

    merge_time = end - start
    merge_times.append(merge_time)

    print(f"{size:<15}{bubble_time:<20.6f}{merge_time:<20.6f}")
```

Output

```
============================================================
Algorithm Comparison: Bubble Sort vs Merge Sort
============================================================
Input Size     Bubble Sort         Merge Sort
100            0.000649            0.002206
500            0.075374            0.005322
1000           0.135254            0.005223
2000           0.249198            0.004304
```

Complexity Analysis

Feature	Bubble Sort	Merge Sort
Time Complexity	$O(n^2)$	O(n log n)
Best Case	O(n)	O(n log n)
Average Case	$O(n^2)$	O(n log n)
Worst Case	$O(n^2)$	O(n log n)
Space Complexity	O(1)	O(n)



Analysis

- Bubble Sort repeatedly compares adjacent elements, resulting in many comparisons and swaps.
- Merge Sort divides the array into smaller parts and merges them efficiently.
- Therefore, Merge Sort requires fewer operations for large inputs.

Conclusion

- Bubble Sort and Merge Sort were compared using the same dataset.
- Bubble Sort has **$O(n^2)$** time complexity, making it slower for large datasets.
- Merge Sort has **O(n log n)** time complexity and performs efficiently even with large inputs.
- The graph and execution time results clearly show that **Merge Sort outperforms Bubble Sort**.
- Therefore, Merge Sort is the preferred choice for sorting large datasets.

References

1. Cormen, T. H., *Introduction to Algorithms* (MIT Press).
2. Anany Levitin, *Introduction to the Design and Analysis of Algorithms*.
3. Python Official Documentation – https://docs.python.org/
4. GeeksforGeeks – Sorting Algorithms.
5. Google Colab Documentation.

THANK YOU